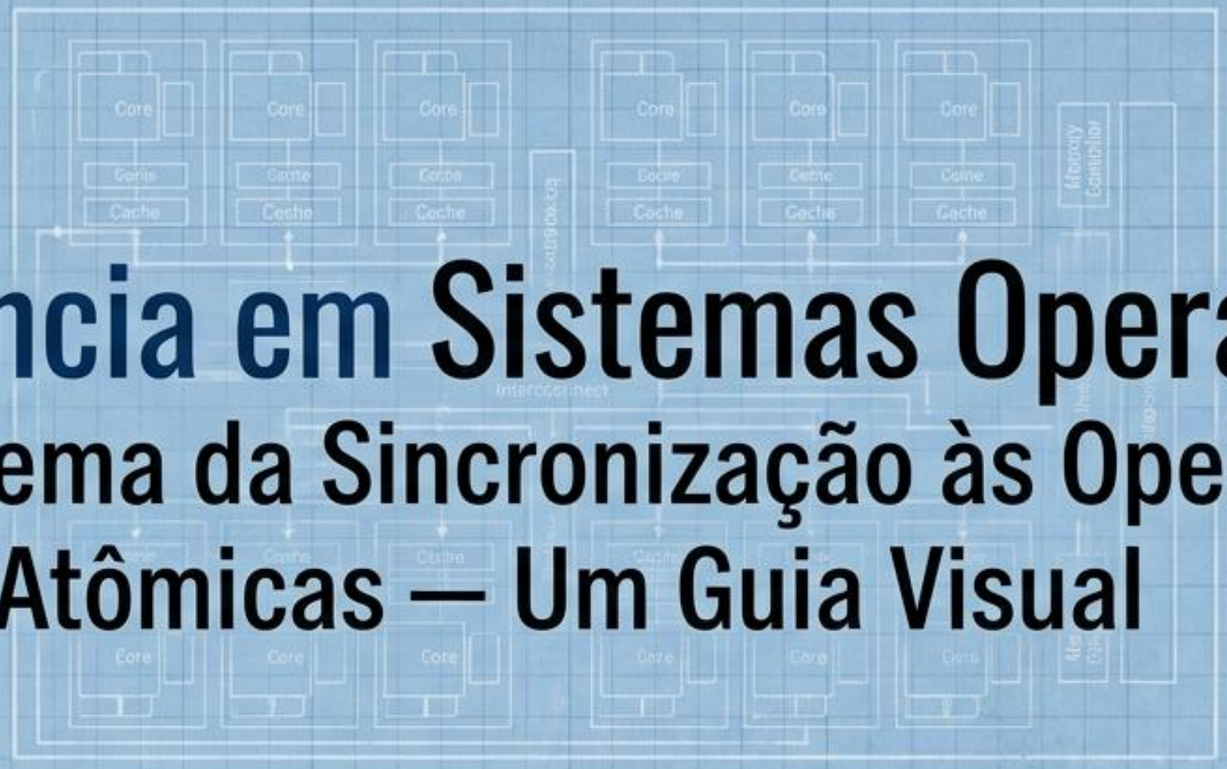


Concorrência em Sistemas Operacionais

Do Problema da Sincronização às Operações Atômicas — Um Guia Visual

Universidade do Estado de Mato Grosso (UNEMAT) – Campus de Alto Araguaia

Curso: Bacharelado em Ciência da Computação

Disciplina: Sistemas Operacionais

Professor: Prof. Me. Juvenal Neto

Base Bibliográfica:
Fundamentado nos clássicos da computação:

- Silberschatz
- Tanenbaum
- Stallings

Gemini Notebook



Caixa Eletrônico A

Depósito: + R$ 100



Depósito: + R$ 100



App Mobile B

CONTA BANCÁRIA CENTRAL

Saldo Inicial: R$ 1.000

Cálculo Lógico Esperado:

R$ 1.000 + R$ 100 (A) + R$ 100 (B) = R$ 1.200

Resultado Real (A Anomalia):

R$ 1.100

⚠ Um depósito desapareceu.

Como a precisão matemática da máquina pode falhar em uma simples adição?
A resposta reside na arquitetura da Concorrência.

Gemini Notebook

Concorrência em 1 Núcleo (Time Slicing)



Múltiplas atividades progridem em períodos sobrepostos.
O SO alterna rapidamente entre elas, criando a ilusão de simultaneidade.

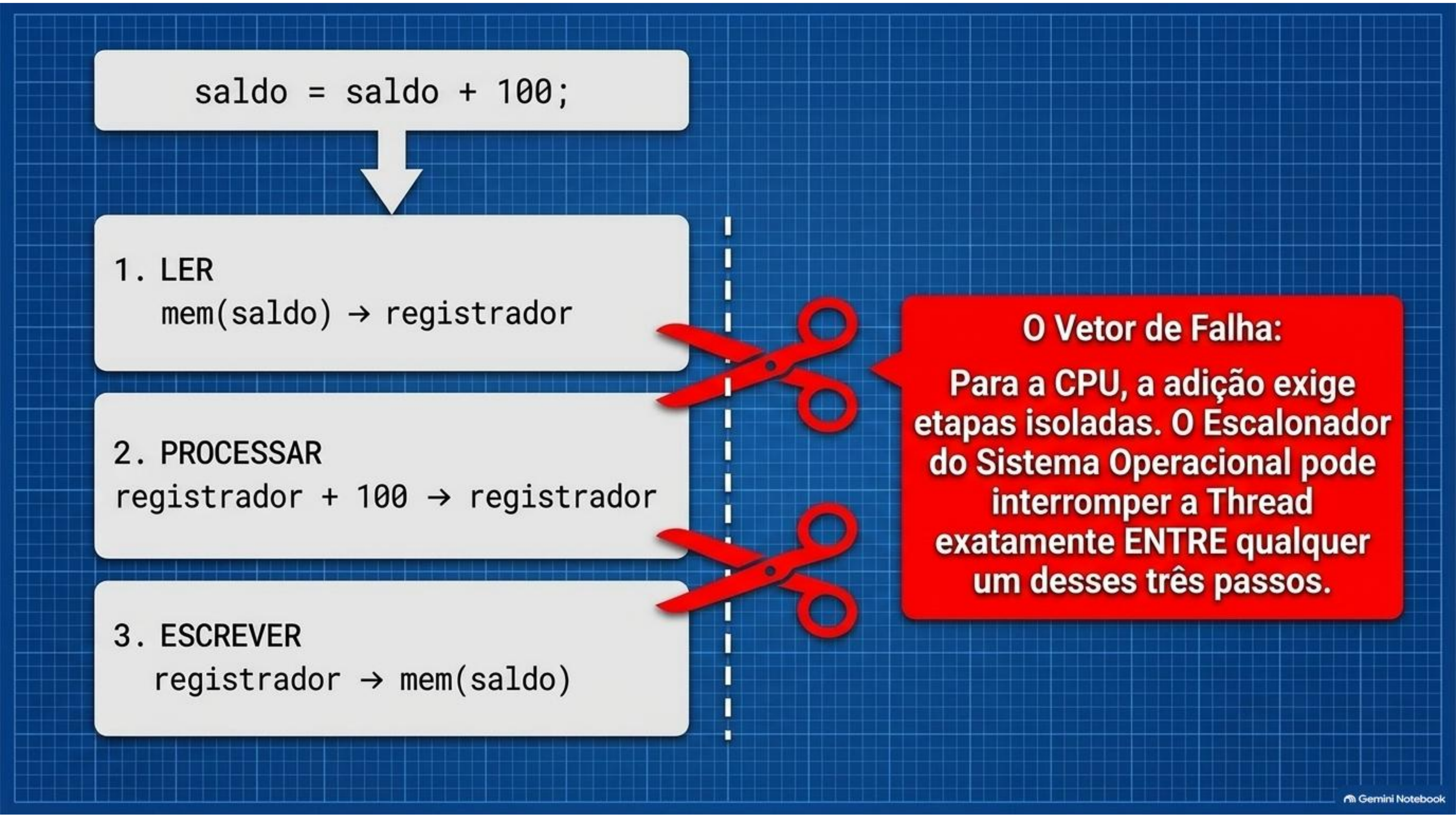
Paralelismo em Múltiplos Núcleos



Múltiplas atividades executam literalmente no mesmo instante físico (requer múltiplos núcleos).

Alerta de Arquitetura: Falhas de sincronização ocorrem em AMBOS os cenários devido ao compartilhamento da mesma memória.

Gemini Notebook



Gemini Notebook

Exemplo de Condição de Corrida: Atualização Perdida

Tempo	Thread A (T1)	Thread B (T2)	Estado da Memória
T0	Lê Saldo (1000)		1000
T1	[Interrompida]	Lê Saldo (1000)	1000
T2		Soma 100, Escreve	1100
T3	Soma 100 (ao valor antigo)	[Interrompida]	1100
T4	Escreve		1100

Atualização Perdida: O T1 reescreveu a memória com base em uma leitura desatualizada, obliterando o depósito do T2.

Condição de Corrida (Race Condition): Ocorre quando o resultado final depende da ordem temporal exata (interleaving) das operações concorrentes desprotegidas.

Gemini Notebook

```
1  void processar_deposito(int valor) {
2
3      validar_conta();
4
5
6
7
8      // Área de Risco
9      saldo = saldo + valor;
10     registros_banco.update();
11
12
13     registrar_log_acesso();
14 }
```



O Alvo: Região Crítica

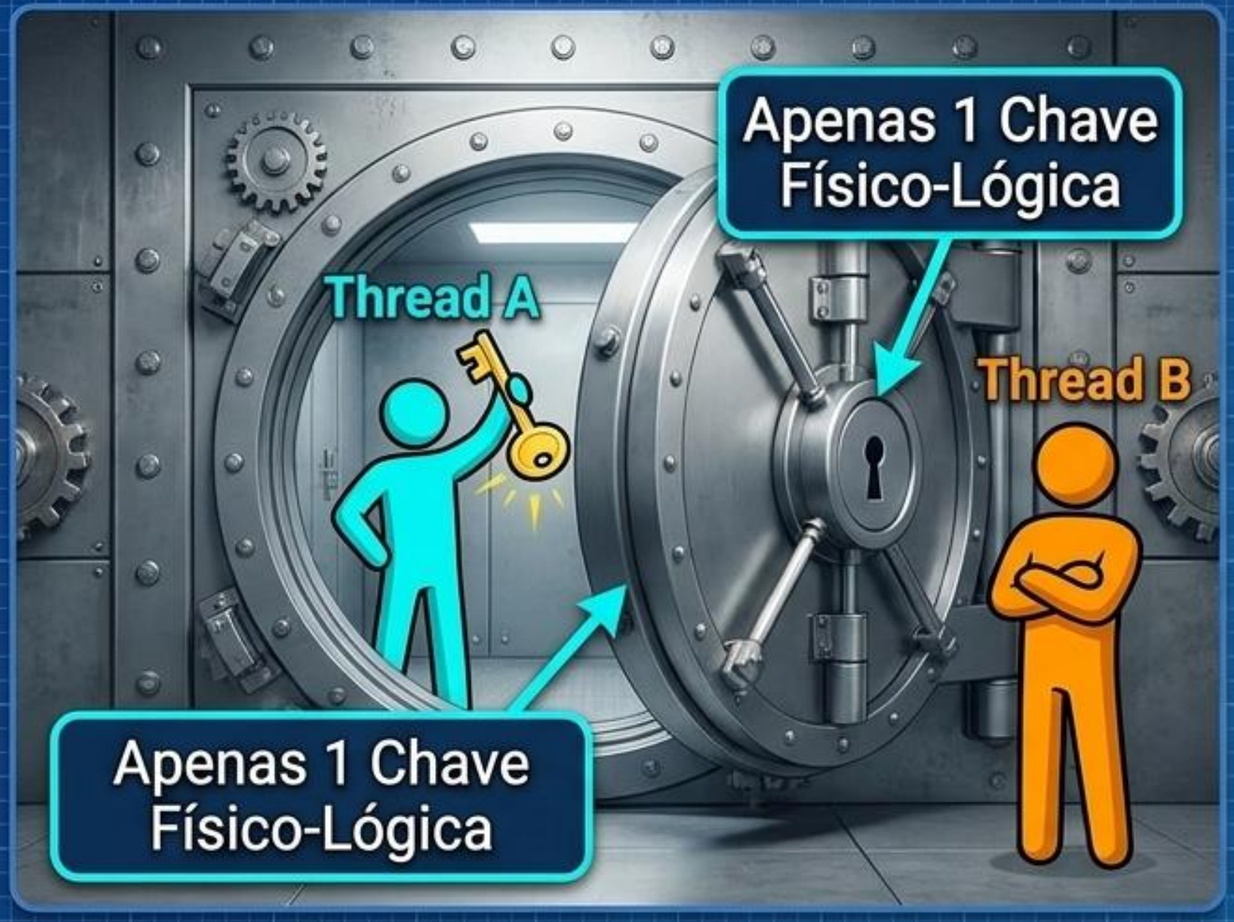
Onde exatamente o problema reside?

Definição: É o trecho específico do código onde a tarefa acessa ou modifica recursos, variáveis ou arquivos que são COMPARTILHADOS com outras tarefas.

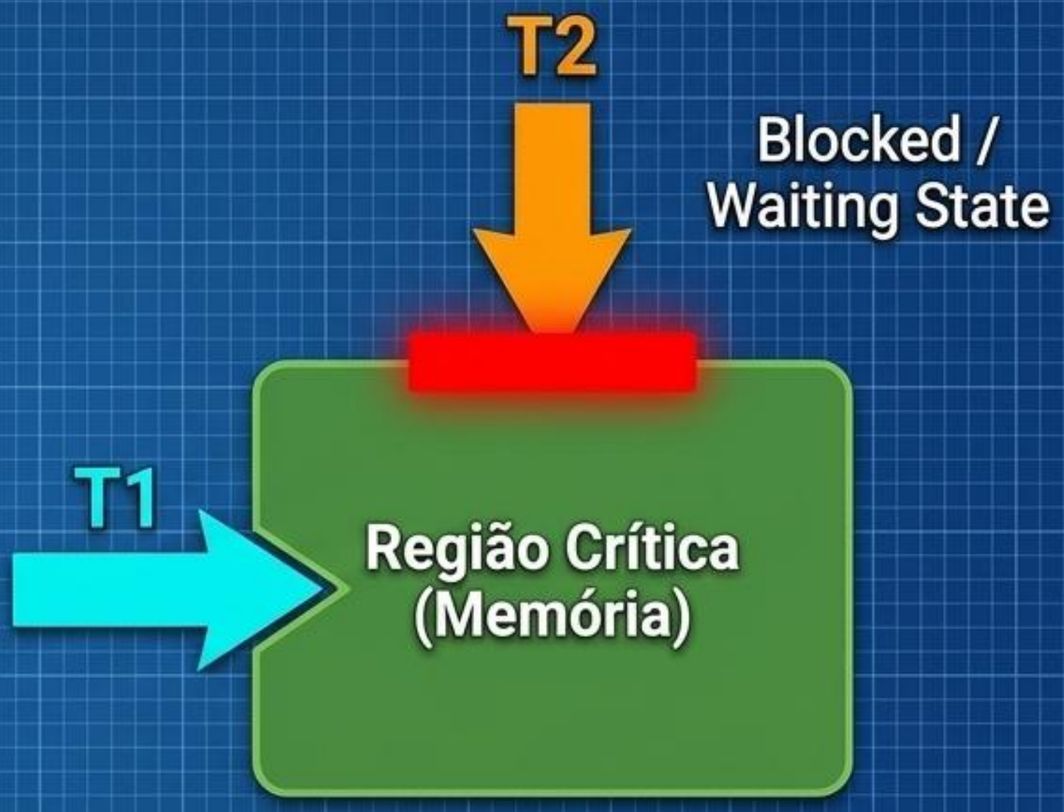
A Regra Arquitetural: A execução entrelaçada de regiões críticas compromete a consistência estrutural dos dados. O objetivo do Sistema Operacional é isolar permanentemente esta área.

Gemini Notebook

A Metáfora



O Sistema



A Solução Lógica: Exclusão Mútua

O princípio garantidor de que apenas UMA atividade pode estar executando sua respectiva Região Crítica por vez. Se a **Thread A** está manipulando o dado, a **Thread B** **fica terminantemente bloqueada** e deve aguardar a liberação do recurso.

Gemini Notebook

Os 3 Requisitos para uma Solução Correta



1. Exclusão Mútua

Apenas um processo por vez é permitido dentro da região crítica. Sem exceções.



2. Progresso

Se a região está livre, a decisão de qual thread entrará não pode ser adiada indefinidamente. Apenas threads interessadas participam da decisão.



3. Espera Limitada

Deve existir um teto rigoroso de quantas vezes outras threads podem 'furar a fila' após uma solicitação. Evita a inanição (starvation).

Gemini Notebook

A Falha do Software Ingênuo (Naive Lock)

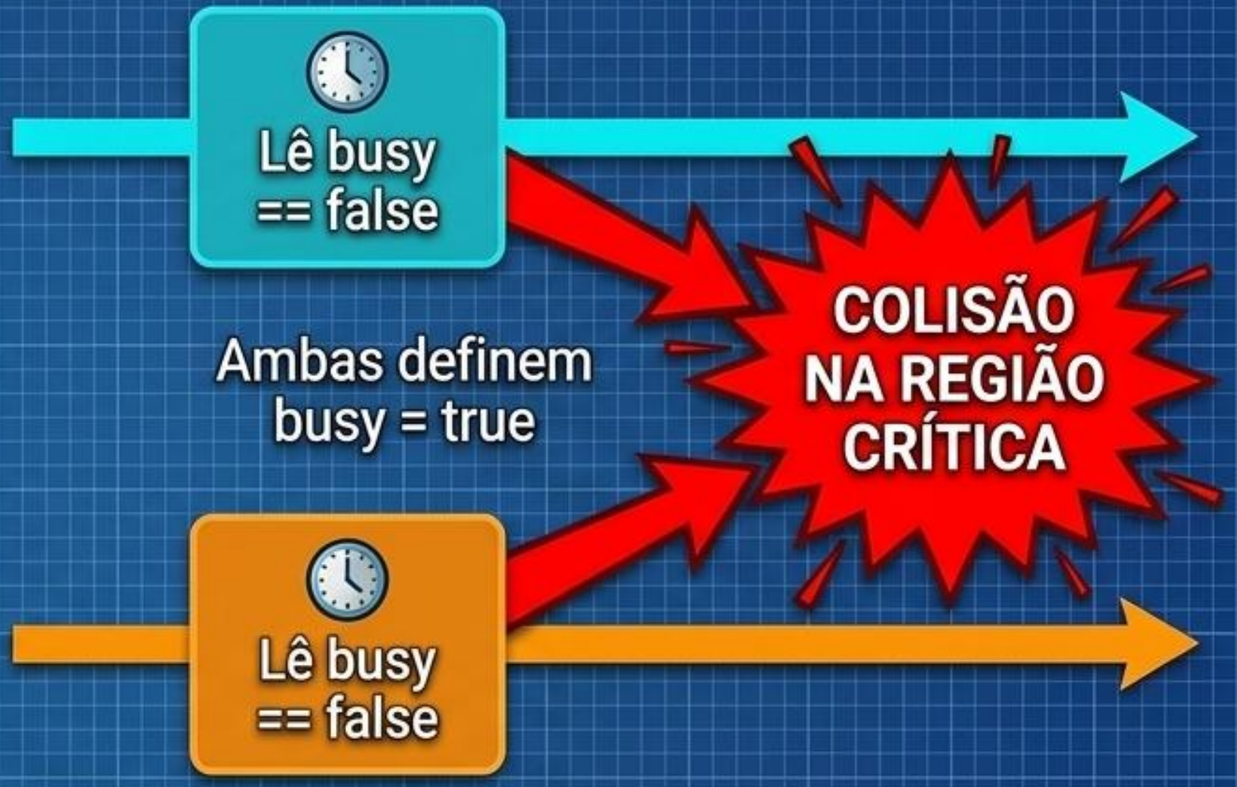
```
while (busy) { espere };

busy = true;

[ Região Crítica ]

busy = false;
```

Ponto de Falha Crítica



A Insuficiência do Software Comum:

Por que não usar apenas um sinalizador booleano? Porque o próprio ato de testar a variável e depois alterá-la compõe uma Região Crítica! Se duas threads testarem a condição no exato mesmo nanossegundo, ambas acharão que a porta está aberta. Precisamos do auxílio do hardware.

Gemini Notebook

O Socorro do Hardware: Operações Atômicas

Instrução Normal



Pode ser fatiada pelo Escalonador (Interrupção).

Operação Atômica



Do grego *atomos* (indivisível). Indestrutível pelo Escalonador.

São instruções especiais providas fisicamente pela arquitetura do processador. Uma vez iniciada, a CPU garante que a instrução ocorra de forma contínua e ininterrupta desão. Nenhuma troca de contexto pode ocorrer durante sua execução.

Gemini Notebook

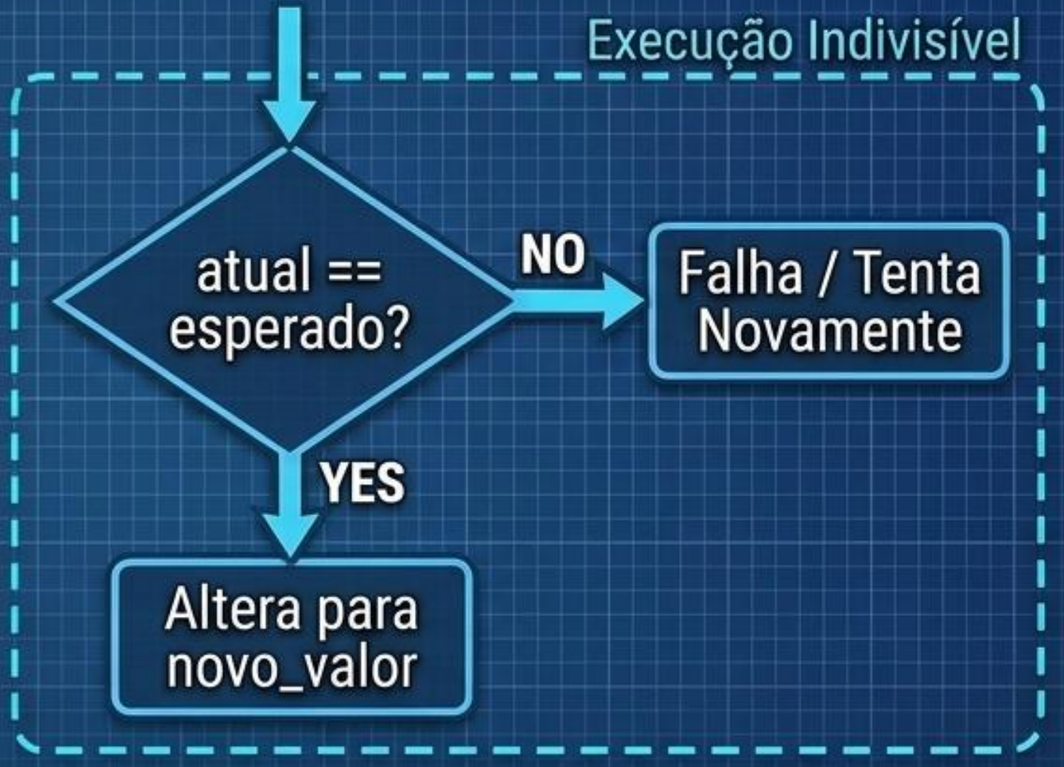
Mecanismos Atômicos na Prática

Test-and-Set (TSL)



Lê o estado da trava e, no mesmo pulso elétrico, escreve que ela está trancada.

Compare-and-Swap (CAS / XCHG)



Compara o valor atual e, se for o esperado, permuta para o novo valor. Base dos Spinlocks modernos.

Nota: A atomicidade resolve a instrução do hardware, mas o programador ainda deve projetar a sincronização corretamente na lógica do código.

Gemini Notebook

A Linha do Tempo Curada (Com Exclusão Mútua)

Tempo	Thread A (T1)	Thread B (T2)	Estado da Memória
T0	**[LOCK Atômico]** Lê (1000)		1000
T1	[Interrompida]	Tenta **[LOCK] -> FALHA** (Bloqueada)	1000
T2	Soma 100, Escreve (1100). **[UNLOCK]**	[Aguardando...]	1100
T3		**[LOCK Atômico]** Lê (1100)	1100
T4		Soma 100, Escreve (1200). **[UNLOCK]**	**1200**

A precisão matemática é restaurada: R$ 1.000 + R$ 100 + R$ 100 = R$ 1.200. Este é o alicerce absoluto por trás de Bancos de Dados (ACID), Servidores Web (Apache) e Kernels Multiprocessados (Linux/Windows).

Gemini Notebook

Matriz de Desambiguação

Conceito	Papel na Arquitetura	Definição Técnica Rigorosa
Concorrência	O Cenário	Múltiplas atividades em progresso disputando recursos na mesma linha de tempo.
Condição de Corrida	O Risco / Bug	Falha onde o resultado dos dados é corrompido pela ordem temporal exata das execuções sobrepostas.
Região Crítica	O Alvo	O perímetro exato de código que manipula a memória compartilhada e exige proteção urgente.
Exclusão Mútua	A Regra (Lógica)	Princípio arquitetural que proíbe duas ou mais regiões críticas de operarem no mesmo instante.
Operação Atômica	A Ferramenta (Física)	Instrução de hardware irredutível e indivisível usada para construir a fechadura da exclusão mútua.

Gemini Notebook

A Síntese Arquitetural



A concorrência fornece poder aos Sistemas Operacionais, mas cobra o alto preço da coordenação. A compreensão desta cadeia causal separa programadores amadores de verdadeiros Engenheiros de Software.

Próximos Passos: Construção de Mutexes, Semáforos e Monitores.

Gemini Notebook

Exercício de Fixação: Diagnóstico de Sistema

```
int itens = 10;

Thread A {
    itens = itens - 1;
}

Thread B {
    if (itens > 0)
        processar(itens);
}
```

Questão 1: Onde está a **Região Crítica** neste código?

Diagnóstico: Toda linha onde a variável compartilhada **'itens'** é lida, testada (no **if**) ou modificada simultaneamente.

Questão 2: Existe risco de Condição de Corrida? Justifique.

Diagnóstico: Sim! Se a Thread B testa **itens > 0** (lê 10), é interrompida, e a Thread A reduz para 9 (ou 0), a Thread B pode processar um item usando um estado desatualizado. Ambas exigem isolamento por Exclusão Mútua atômica.

Gemini Notebook